

ATIVIDADE AVALIATIVA - WARMUP DO PROJETO

Engenharia de Software II

Felipe Avalon
Gabriel Soares
Luis Otavio Albergoni

1. Escolha de uma funcionalidade principal

Alertas automáticos de vencimento de certificados

2. Descrição do fluxo completo

Funcionalidade: Alertas de certificados próximos do vencimento

Descrição geral do funcionamento

Um certificado é cadastrado no sistema com uma data de vencimento.

Os usuários utilizam normalmente o sistema, sem precisar acionar manualmente essa funcionalidade.

O próprio sistema é responsável por verificar diariamente os certificados cadastrados.

Essa verificação é feita automaticamente por uma rotina (scheduler).

Se o tempo restante para expiração do certificado for menor ou igual aos prazos definidos (60, 30 ou 7 dias), o sistema deve gerar uma notificação.

Essa notificação precisa ser visível para o usuário no sistema (dashboard) e também enviada por e-mail.

A notificação permanece visível até que o usuário visualize ou interaja com ela.

quem inicia a ação

O sistema (através de uma rotina automática agendada - scheduler)

o que acontece primeiro

O scheduler executa diariamente e verifica a data de vencimento dos certificados cadastrados no banco de dados.

quais partes do sistema são acionadas

Scheduler (rotina automática)

Service de alertas (regra de negócio)

Repository de certificados (busca no banco)

Repository de alertas (registro dos alertas)
Serviço de e-mail
Sistema de notificações (dashboard)

como os dados circulam

O scheduler inicia o processo e solicita ao sistema a verificação dos certificados.

O sistema acessa o banco de dados através do repository de certificados e obtém a lista de certificados com data de vencimento.

Esses dados são enviados para a camada de service, que calcula o tempo restante para cada certificado.

Se o tempo restante estiver dentro dos limites definidos (60, 30 ou 7 dias), o service verifica se já existe alerta para aquele certificado.

Caso não exista, o sistema cria um novo alerta e salva no banco através do repository de alertas.

Depois disso, o sistema envia os dados do alerta para:

o serviço de e-mail (para envio ao responsável)

o sistema de notificações (para exibir no dashboard)

Se ocorrer erro no envio do e-mail, o sistema registra a falha e tenta novamente depois.

qual é o resultado final

O usuário recebe um alerta informando que o certificado está próximo do vencimento.

Esse alerta aparece no dashboard do sistema e também pode ser enviado por e-mail.

O usuário consegue visualizar o certificado e tomar ação antes do vencimento.

3. Mapeamento técnico

Na funcionalidade de alertas automáticos de vencimento, as principais classes que participam do fluxo são Certificado, GestaoCertificado, Usuário, Notificação e LogAuditoria, conforme definido no modelo de análise do projeto.

A classe Certificado é responsável por armazenar os dados do certificado, como nome, datas e status, sendo a principal fonte de informação para verificar o vencimento.

A classe GestaoCertificado concentra a lógica de negócio, realizando a verificação das datas e decidindo quando um alerta deve ser gerado (60, 30 ou 7 dias antes do vencimento).

A classe Notificação representa o alerta gerado pelo sistema, sendo responsável por armazenar as informações que serão exibidas ao usuário, como mensagem, nível de criticidade e status (visualizado ou não).

A classe Usuário representa quem recebe ou visualiza o alerta, tanto no sistema quanto por e-mail.

Já a classe LogAuditoria é responsável por registrar os eventos, garantindo histórico e rastreabilidade das ações realizadas.

A comunicação entre essas classes ocorre de forma simples:

Sistema → GestaoCertificado → Certificado → Notificação → Usuário → LogAuditoria.

4. Organização arquitetural

Para a funcionalidade de alertas automáticos de vencimento, o sistema foi organizado nas camadas: routes, controllers, middlewares, validators, services e models, com separação clara de responsabilidades.

Onde cada parte do fluxo está localizada

Routes: definem as rotas do sistema e direcionam as requisições.

Controllers: recebem a requisição e chamam os services.

Middlewares: fazem autenticação e controle de acesso.

Validators: validam os dados (ex: data de vencimento).

Services: executam a regra de negócio (verificar vencimento e gerar alertas).

Models: representam os dados no sistema (Certificado, Usuário, Notificação, LogAuditoria).

Classes por camada

Routes: certificateRoutes, notificationRoutes

Controllers: CertificateController, NotificationController

Middlewares: AuthMiddleware

Validators: CertificateValidator

Services: GestaoCertificado, NotificacaoService

Models: Certificado, Usuário, Notificação, LogAuditoria

Como ocorre a comunicação

Usuário/Sistema → Routes → Middlewares → Validators → Controllers → Services → Models

No caso dos alertas automáticos:

Sistema → GestaoCertificado → Certificado → Notificação → Usuário → LogAuditoria

Mistura de responsabilidades

A separação está bem definida, porém a classe GestaoCertificado concentra bastante lógica, podendo acumular responsabilidades.

Justificativa

Essa estrutura foi adotada por ser simples, organizada e facilitar a separação entre validação, controle e regra de negócio.

5. Identificação de problemas

De forma geral, o fluxo proposto faz sentido, pois segue uma lógica simples: o sistema verifica os certificados diariamente e gera notificações.

Não parece haver acoplamento indevido nas camadas.

Porém, a classe GestaoCertificado pode acumular responsabilidades excessivas.

Também falta uma separação melhor da parte de notificação.

Uma melhoria seria dividir melhor as responsabilidades para facilitar manutenção futura.

6. Definição de classes, caso necessário

As principais entidades do domínio seriam:

Certificado

Usuário

Notificação

LogAuditoria

A classe Notificação passa a ser responsável por armazenar as informações do alerta, como mensagem, data, tipo e status.

Sua inclusão melhora a organização do sistema e a separação de responsabilidades.